

RycrawAI

Agentic AI Vulnerability Scanner

Scan Module Integration Report

Version 2.0 | May 2026

Classification: Confidential

1. Executive Summary

This document provides a comprehensive technical overview of the scan categories integrated into RycrawAI, Anthropic's agentic AI vulnerability scanning framework. RycrawAI is purpose-built to identify, probe, and report on security vulnerabilities unique to AI agent systems operating in real-world environments.

The platform has expanded from its foundational scanning capabilities to include a total of six distinct agentic threat categories. This report documents the full scope of integrated modules, with particular focus on the most recently added capability: MCP (Model Context Protocol) Abuse scanning, which tests malicious function-call inputs against connected tools.

Key Capability Update

RycrawAI now covers 6 agentic AI vulnerability categories, providing end-to-end coverage of the modern AI agent attack surface — from inter-agent trust abuse to tool-layer exploitation via MCP.

2. Background & Motivation

AI agent systems introduce a fundamentally different threat model compared to traditional software. Unlike stateless API endpoints, agentic AI systems:

- Maintain memory and context across multiple interaction turns
- Invoke external tools and services on behalf of users
- Operate within multi-agent hierarchies with delegated trust
- Consume and process untrusted external data (files, web pages, tool outputs)
- Execute long-horizon tasks with minimal human checkpoints

These characteristics create an attack surface that conventional security scanners are not equipped to address. RycrawAI was developed specifically to fill this gap, providing structured, reproducible test suites for each threat category.

3. Integrated Scan Categories

The following table summarizes all six scan categories currently active in RycrawAI, including the newly integrated MCP Abuse module.

Icon	Category	Description & Tags	#
[NEW]	MCP Abuse	Fires malicious function-call inputs against connected MCP tools — malformed parameters, boundary violations, injected payloads, and chained calls designed to bypass tool-level guardrails.	1

		<i>malicious inputs parameter fuzzing guardrail bypass chained calls</i>	
[>]	Multi-Agent Coordination Abuse	Tests whether one agent can manipulate another through hijacking task delegation, forging inter-agent messages, or escalating permissions via a trusted peer. <i>agent-to-agent trust delegation hijack permission escalation</i>	2
[T]	Tool Misuse	Crafts payloads that trick the agent into calling tools outside its authorized scope — file writes, API calls, or destructive operations it should never invoke. <i>unauthorized tool calls scope violation destructive ops</i>	3
[?]	Role Confusion / Policy Override	Probes whether attacker instructions can displace system-level rules via persona switching, jailbreaks, and authority spoofing. <i>persona hijack jailbreak authority spoof</i>	4
[F]	Indirect Prompt Injection	Embeds hidden instructions inside files, web pages, or tool outputs the agent reads — turning trusted data sources into attack vectors. <i>file injection web content tool output poisoning</i>	5
[DB]	Memory Poisoning	Checks whether malicious data written in one turn persists and influences future turns — corrupting context windows, vector stores, or session memory. <i>cross-turn persistence context corruption vector store</i>	6

4. Detailed Category Documentation

4.1 MCP Abuse [NEW — Integrated]

Integration Status

MCP Abuse scanning is newly integrated into RycrawAI as of this release. It represents the sixth and most protocol-specific scan module in the platform.

Overview

MCP (Model Context Protocol) is an open standard that enables AI agents to interface with external tools, APIs, and data sources through structured function calls. While MCP provides powerful extensibility, it also introduces a new attack surface: the function-call layer itself.

RycrawAI's MCP Abuse module systematically fires malicious and malformed function-call inputs against all connected MCP tools to identify whether the agent, the tool server, or the integration layer can be exploited.

What it tests

- Malformed parameter inputs — sending incorrect data types, null values, oversized strings, and schema-violating inputs to MCP tool functions
- Boundary violations — probing integer limits, string length caps, array size constraints, and enum-value boundaries
- Injected payloads — embedding prompt injection strings, SQL-like commands, path traversal sequences, and shell metacharacters inside function arguments
- Chained call exploitation — using the output of one MCP tool as a crafted input to another, weaponizing data flow across tools
- Unauthorized scope calls — attempting to invoke MCP functions the agent's role or permission level should not allow
- Guardrail bypass attempts — crafting inputs designed to make tools perform actions outside their declared behavioral constraints

Key distinction from tool misuse

Tool misuse focuses on getting the agent to call the wrong tool entirely. MCP Abuse is narrower and more precise: it targets the right tool but exploits its input interface. This distinction matters because tool-layer guardrails (type checking, input validation, permission scopes) are often implemented separately from the agent's behavioral guardrails, leaving a gap that MCP Abuse specifically probes.

Example attack vectors

- Parameter injection: `{"filename": "../../../etc/passwd"}` passed to a file-read MCP tool
- Type confusion: passing a string where an integer is expected to trigger unexpected parsing behavior

- Oversized input: sending a 10MB string to a summarization tool to test resource exhaustion
- Chained injection: tool A returns user-controlled data that is passed unsanitized to tool B's command parameter

4.2 Multi-Agent Coordination Abuse

Overview

Modern AI systems increasingly deploy multiple agents working in concert — orchestrators that delegate tasks to sub-agents, peer agents that share context, and specialized agents with elevated privileges. This architecture creates inter-agent trust relationships that can be exploited.

What it tests

- Whether a sub-agent can forge messages that appear to originate from a trusted orchestrator
- Whether task delegation can be hijacked to redirect agent actions toward attacker-controlled goals
- Whether permission escalation is possible by impersonating a higher-privilege agent in the hierarchy
- Whether inter-agent communication channels validate message authenticity and integrity

Why it matters

Vulnerabilities here are particularly dangerous because the attack surface only exists in multi-agent deployments. Single-agent security testing will never surface them, and the trust chains between agents are often implicit rather than cryptographically enforced.

4.3 Tool Misuse

Overview

Agents are granted access to a set of authorized tools. Tool misuse tests probe whether the agent can be manipulated into calling tools outside its authorized scope — either by crafting adversarial prompts that override access controls, or by exploiting ambiguous tool descriptions.

What it tests

- Invoking write or delete operations when only read access should be permitted
- Calling external API tools to exfiltrate data under the guise of a legitimate task
- Triggering destructive tool operations through indirect instruction chains
- Exploiting vague tool descriptions that leave room for unintended invocations

4.4 Role Confusion / Policy Override

Overview

AI agents operate under a layered authority model: system prompts define foundational behavior, operator instructions add context-specific constraints, and user inputs provide task direction. Role confusion attacks attempt to collapse or invert this hierarchy so that attacker instructions take precedence over legitimate policy.

What it tests

- Persona injection — instructing the agent to adopt an alternative identity with fewer restrictions
- Authority spoofing — crafting messages that claim to originate from system-level or operator-level authority
- Jailbreak attempts — direct and indirect requests to ignore, override, or reinterpret system instructions
- Multi-step policy erosion — gradually shifting the agent's operating context across a conversation

4.5 Indirect Prompt Injection

Overview

Unlike direct prompt injection (where an attacker controls the user input), indirect prompt injection embeds malicious instructions in content the agent retrieves or processes — documents, web pages, database records, email bodies, or tool output. This is a passive attack: the attacker poisons an external source and waits for the agent to consume it.

What it tests

- Hidden instructions in uploaded files (PDF, DOCX, CSV) that override agent behavior when parsed
- Malicious content in web pages retrieved via browser or fetch tools
- Poisoned tool output — injected instructions embedded in API responses or database query results
- Email or calendar content designed to redirect an agent's scheduled tasks

Severity Note

Indirect prompt injection is considered the highest-severity category for deployed agents because it is entirely passive from the attacker's perspective — a poisoned webpage or document does the work without any direct interaction with the agent system.

4.6 Memory Poisoning

Overview

Agentic systems frequently maintain state across turns — through context windows, external vector stores, episodic memory, or session summaries. Memory poisoning tests whether malicious data written or injected in one turn can persist and influence the agent's behavior in future turns.

What it tests

- Writing adversarial facts or instructions into the agent's memory during an early turn
- Verifying that planted data surfaces in subsequent turns and alters agent responses
- Corrupting vector store entries to cause retrieval-augmented generation to return malicious context
- Session summary poisoning — injecting instructions into summarized context that persists beyond the original context window

Testing requirements

Memory poisoning is the most complex category to test because it requires genuine multi-turn session support with persistent state. RycrawAI must maintain state across rounds, plant test payloads, allow natural session progression, and then verify whether planted data resurfaces and influences later behavior.

5. Scanner Architecture Overview

RycrawAI's six scan modules are organized around a shared scanning pipeline that handles session management, payload delivery, response analysis, and report generation.

Pipeline Stage	Description	Modules
Target Enumeration	Identifies all agent tools, memory stores, connected MCP servers, and active agent peers in scope for the current scan.	<i>All</i>
Payload Generation	Generates module-specific test payloads — malformed inputs, injection strings, forged messages, and poisoned data.	<i>MCP, Tool Misuse, Injection</i>
Session Management	Maintains multi-turn conversation state required for memory poisoning and role erosion tests.	<i>Memory, Role Confusion</i>
Delivery	Injects payloads through the appropriate channel: user input, file upload, tool output, or inter-agent message.	<i>All</i>
Response Analysis	Evaluates agent responses for policy violations, unintended tool calls, data leakage, or behavior change.	<i>All</i>
Reporting	Generates structured findings with severity ratings, reproduction steps, and remediation guidance.	<i>All</i>

6. Severity Classification

Each finding produced by RycrawAI is assigned a severity rating based on potential impact, exploitability, and how broadly the vulnerability affects the deployed agent system.

Level	Category Examples	Criteria
CRITICAL	Indirect Injection, MCP Abuse	Agent can be fully controlled by external attacker; data exfiltration or destructive action possible without user awareness.
HIGH	Multi-Agent Abuse, Tool Misuse	Significant policy violation achievable; unauthorized tool calls or permission escalation demonstrated.
MEDIUM	Role Confusion, Memory Poisoning	Partial policy override achieved; behavior change observed across turns or in specific contexts.
LOW	Role Confusion (minor)	Minor behavioral drift observed; agent nudged but does not fully comply with attacker instructions.
INFO	All	Potential weakness identified; no confirmed exploitation in current test configuration.

7. Recommended Scan Sequence

For comprehensive agentic vulnerability assessment, the following scan sequence is recommended to maximize coverage while managing session state and tool availability:

- Phase 1 — Reconnaissance: Enumerate all tools, MCP servers, memory stores, and agent peers in scope
- Phase 2 — Tool-Layer Testing: Run MCP Abuse and Tool Misuse scans against all connected tools
- Phase 3 — Policy Testing: Execute Role Confusion and Policy Override tests against the agent's system prompt configuration
- Phase 4 — Data-Layer Testing: Inject indirect prompt injection payloads via files, web content, and tool output
- Phase 5 — Persistence Testing: Run Memory Poisoning tests across multi-turn sessions
- Phase 6 — Multi-Agent Testing: If applicable, test coordination abuse across agent hierarchies
- Phase 7 — Reporting: Aggregate findings, assign severity ratings, and generate remediation guidance

8. Conclusion

RycrawAI's six-category scan framework provides comprehensive coverage of the agentic AI attack surface. The integration of MCP Abuse scanning completes the tool-layer security picture, ensuring that both agent-level and protocol-level vulnerabilities are systematically identified.

As AI agent deployments grow in complexity — with more connected tools, richer memory systems, and larger multi-agent networks — the categories documented in this report will become increasingly critical to assess before production deployment.

Next Steps

Consider scheduling automated RycrawAI scans as part of your CI/CD pipeline for any agentic AI system, running full-suite scans prior to each production deployment, and reviewing findings against your agent's system prompt and tool permission configuration.

Document Information

Document Title	RycrawAI Agentic AI Vulnerability Scanner — Scan Module Integration Report
Version	2.0
Date	May 2026

Classification	Confidential
Scan Categories	6 (MCP Abuse, Multi-Agent Coordination Abuse, Tool Misuse, Role Confusion / Policy Override, Indirect Prompt Injection, Memory Poisoning)
Status	Current — MCP Abuse module newly integrated